

SENIOR ENGINEER INTERVIEW PREP

Gokulakonda Gautham

DOCUMENT 9 OF 10

System Design & Architecture Tradeoffs

8 Questions | CAP Theorem | URL Shortener | Notification System | Rate Limiter | Chat System

System Design & Architecture Tradeoffs

Q1: How to approach a system design interview — the framework.

System design interviews are open-ended and ambiguous by design. The interviewer is evaluating your ability to structure ambiguity, ask the right questions, make tradeoff decisions, and communicate at multiple levels of abstraction simultaneously.

Step 1 — Clarify requirements (5 min): Ask: What is the scale? (users, requests/sec, data volume). Read-heavy or write-heavy? Latency requirements? Consistency requirements (can we tolerate stale data)? What features are in scope for today? Geographic distribution needed?

Step 2 — Capacity estimation (3 min): Back-of-envelope math. Shows you think in orders of magnitude. 1M DAU $\times$ 10 actions/day = 10M events/day = $\sim$ 120 events/sec average, $\sim$ 600 peak. Storage: 10M events $\times$ 500 bytes = 5GB/day. These numbers drive your database and infrastructure choices.

Step 3 — High-level design (10 min): Draw the major components: client, API layer, services, databases, caches, queues. Don't go deep yet — get the skeleton right. Walk through a key user flow end-to-end on the diagram.

Step 4 — Deep dive (15-20 min): Interviewer will guide you to the interesting parts. Go deep: how does the DB schema look? How do you handle the write hotspot? What happens when the cache fails? How do you scale the bottleneck?

Step 5 — Identify and address bottlenecks: Every design has a bottleneck. Find it before the interviewer does: single points of failure, scaling limits, consistency gaps, latency hotspots. Propose solutions and discuss their tradeoffs.

```
# Numbers every engineer should know (approximations)
L1 cache: 0.5 ns
L2 cache: 7 ns
RAM:      100 ns
SSD:      100 microseconds
HDD:      10 ms
Network same DC: 0.5 ms
Network cross-region: 50-150 ms

# Throughput baselines
Single DB (Postgres):  $\sim$ 5,000 writes/sec,  $\sim$ 50,000 reads/sec (with index)
Redis:  $\sim$ 100,000 ops/sec single node
Kafka:  $\sim$ 1,000,000 messages/sec per broker
HTTP server (Go/FastAPI): 50,000-200,000 req/sec per instance

# Scale reference
1M DAU:  $\sim$ 120 req/sec avg,  $\sim$ 600 req/sec peak
10M DAU:  $\sim$ 1,200 req/sec avg,  $\sim$ 6,000 req/sec peak
100M DAU:  $\sim$ 12,000 req/sec avg,  $\sim$ 60,000 req/sec peak
```

Q2: CAP theorem — what it actually means and how to apply it in design decisions.

CAP theorem states that a distributed system can guarantee at most two of three properties: Consistency, Availability, Partition Tolerance. Partition Tolerance is not optional in real networks — you must choose between C and A during a network partition.

Consistency (C): Every read receives the most recent write or an error. All nodes see the same data at the same time. Not the same as ACID consistency — this is distributed consistency.

Availability (A): Every request receives a response (not an error), though it may not be the latest data. The system keeps serving even when some nodes are down.

Partition Tolerance (P): The system continues operating despite network partitions (some nodes can't communicate with others). In practice, you MUST tolerate partitions — networks fail. So the real choice is: during a partition, do you return an error (choose C) or return potentially stale data (choose A)?

CP systems: Reject requests during partitions to maintain consistency. Examples: HBase, Zookeeper, etcd, traditional RDBMS with synchronous replication. Use when: financial transactions, inventory counts, leader elections — anywhere stale data causes incorrect business outcomes.

AP systems: Return potentially stale data during partitions. Eventually consistent. Examples: Cassandra, DynamoDB (default), CouchDB, DNS. Use when: social feeds, user profiles, shopping carts, analytics — where brief staleness is acceptable.

```
# Practical application in system design

# Payment service: CP — never show wrong balance
# PostgreSQL with synchronous replication
# During partition: return error rather than stale balance

# User profile service: AP — stale profile for 30s is fine
# DynamoDB or Cassandra
# During partition: serve cached/replica data

# Inventory: DEPENDS on the action
# Read (show stock count): AP — show approximate, refresh async
# Write (decrement on purchase): CP — must be accurate to prevent oversell

# PACELC extension (more nuanced than CAP):
# Even WITHOUT partitions: tradeoff between Latency and Consistency
# Synchronous replication: higher consistency, higher latency
# Async replication: lower latency, eventual consistency
```

Q3: Design a URL shortener — the classic system design problem.

URL shortener is a canonical system design problem. It covers: hashing, database design, caching, redirection, and scaling. Walk through it completely.

Requirements: Functional: shorten a long URL, redirect short URL to original, custom aliases optional. Non-functional: 100M URLs created/day, 10B redirects/day (read-heavy 100:1), < 10ms redirect latency, 5-year data retention.

Capacity: 100M URLs/day = 1200 writes/sec. 10B redirects/day = 115,000 reads/sec. Storage: $100M \times 365 \times 5 = 182B$ URLs $\times$ 500 bytes = ~90TB over 5 years (manageable with sharding).

Short URL generation: Need a 7-character code from [a-z, A-Z, 0-9] = 62 chars. $62^7 = 3.5$ trillion combinations — enough. Options: (1) Hash the long URL (MD5/SHA256, take first 7 chars) — collision risk. (2) Base62-encode an auto-incrementing ID — predictable but simple. (3) Pre-generate a pool of random codes.

```
# Architecture
```

```
Client -> CDN -> Load Balancer -> API Servers (stateless)
                                   |
                                   +-----+-----+
                                   |               |
                               Cache (Redis)    DB (PostgreSQL)
                               TTL: 24h for hot URLs   Source of truth
```

```
# DB Schema
CREATE TABLE urls (
  id          BIGSERIAL PRIMARY KEY,
  short_code  VARCHAR(10) UNIQUE NOT NULL,
  long_url    TEXT NOT NULL,
  user_id     BIGINT,
  created_at  TIMESTAMPTZ DEFAULT NOW(),
```

```

    expires_at TIMESTAMPTZ,
    click_count BIGINT DEFAULT 0
);
CREATE INDEX idx_short_code ON urls(short_code); -- primary lookup

# Redirect flow (115k req/sec)
# 1. Check Redis: GET short:<code> -> O(1) ~0.1ms
# 2. Cache hit: return 301/302 redirect — done
# 3. Cache miss: query DB, store in Redis with TTL, redirect
# 99% of traffic served from Redis — DB rarely touched

# 301 vs 302:
# 301 Permanent: browser caches redirect — less server load
# 302 Temporary: browser always hits server — enables analytics
# Use 302 if you need click tracking

# Write flow (1200/sec — trivial for Postgres)
# 1. Generate short code (base62 of DB ID)
# 2. INSERT into DB
# 3. Store in Redis
# 4. Return short URL

```

Scaling writes: 1200 writes/sec is trivial for PostgreSQL. If it weren't: shard by short_code prefix, use Cassandra for write-heavy workloads, or pre-generate short codes in batches (each server gets a range of IDs from a central coordinator — no per-write coordination).

Analytics: Tracking every click synchronously adds latency to the redirect. Instead: publish a click event to Kafka asynchronously, return the redirect immediately. Analytics consumers aggregate in the background. Kafka absorbs click spikes.

Q4: Design a notification system — push, email, SMS at scale.

Notification systems are interesting because they combine fan-out (one event → millions of notifications), reliability (every notification must be delivered), and heterogeneous delivery channels.

Requirements: Support push (iOS/Android), email, SMS. 10M users. Events: new follower, new message, promotional campaign (can send to all 10M). Delivery guarantee: at-least-once. User preferences: opt-out per channel, quiet hours, frequency limits.

Architecture: Event producers publish to Kafka. A Notification Orchestrator consumes events, looks up user preferences, and routes to channel-specific queues. Channel workers (push-worker, email-worker, sms-worker) consume their queues and call third-party providers.

```

# Notification flow
Event Source (order placed, new follower)
  -> Kafka topic: 'notification-triggers'
  -> Notification Orchestrator:
      1. Load user notification preferences (Redis cache)
      2. Check quiet hours, frequency limits
      3. For each enabled channel, publish to channel queue
  -> Push Queue      -> Push Worker  -> FCM / APNs
  -> Email Queue     -> Email Worker -> SendGrid / SES
  -> SMS Queue       -> SMS Worker  -> Twilio

# Schema: notification_preferences
CREATE TABLE notification_preferences (
  user_id    BIGINT PRIMARY KEY,
  push       BOOLEAN DEFAULT TRUE,
  email      BOOLEAN DEFAULT TRUE,
  sms        BOOLEAN DEFAULT FALSE,
  quiet_start TIME DEFAULT '22:00',
  quiet_end  TIME DEFAULT '08:00'
);

```

```
# Delivery tracking
CREATE TABLE notification_log (
  id          UUID PRIMARY KEY,
  user_id     BIGINT,
  channel     VARCHAR(10),
  event_type  VARCHAR(50),
  status      VARCHAR(20), -- queued, sent, delivered, failed
  created_at  TIMESTAMPTZ,
  sent_at     TIMESTAMPTZ
);
```

Fan-out for campaigns: Sending to 10M users: you can't load 10M records and publish 10M messages synchronously. Use a batch job: read users in pages of 1000, publish to Kafka in batches. With 10 partitions and 10 workers, 10M notifications can be dispatched in minutes.

Rate limiting per provider: FCM, SendGrid, Twilio all have rate limits and per-message costs. Use a token bucket per provider to smooth delivery. Queue messages with priority — transactional (OTP, order confirmation) > promotional.

Retry and dead letter: If push delivery fails (device offline), retry 3 times with exponential backoff. After max retries, move to dead-letter queue. DLQ is monitored for alerting. For critical notifications (OTP), try fallback channel (SMS if push fails).

Q5: Horizontal scaling, load balancing, and stateless service design.

Horizontal scaling means adding more instances of a service rather than making a single instance bigger. It requires services to be stateless — any instance must be able to handle any request.

Stateless design: A stateless service stores no session state in memory. Every request is self-contained — includes all auth info (JWT), context (user ID in request), and needed data. State is externalized to Redis (sessions), databases (user data), or object storage (files).

Sticky sessions (anti-pattern): Some systems route users to the same server to preserve in-memory session state. This defeats horizontal scaling — you can't easily rebalance load, and server failure loses sessions. Avoid. Use Redis for shared session storage instead.

Load balancing algorithms: Round-robin: simple, even distribution. Least connections: routes to server with fewest active connections — better for variable request durations. IP hash: same client always routes to same server (useful for WebSocket affinity). Weighted: distribute proportional to server capacity.

```
# Nginx load balancing config
upstream api_servers {
  least_conn; # algorithm
  server api-1:8080 weight=3; # more powerful server
  server api-2:8080 weight=1;
  server api-3:8080 weight=1;
  keepalive 32; # connection pool to upstream
}

# Health checks — remove unhealthy servers
upstream api_servers {
  server api-1:8080;
  server api-2:8080;
  check interval=3000 rise=2 fall=3 timeout=1000;
}

# Kubernetes HPA — autoscale based on CPU
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
spec:
  scaleTargetRef:
    kind: Deployment
    name: api-service
```

```

minReplicas: 3
maxReplicas: 20
metrics:
- type: Resource
  resource:
    name: cpu
    target: { type: Utilization, averageUtilization: 70 }
- type: Pods
  pods:
    metric: { name: http_requests_per_second }
    target: { type: AverageValue, averageValue: '1000' }

```

Database scaling: Vertical: bigger instance (limit ~128 cores). Read replicas: distribute read traffic (eventual consistency). Sharding: partition data across multiple DBs by key (`user_id % N`). CQRS: separate read and write models. Caching: Redis in front reduces DB load by 80-95%.

Q6: Database replication — primary/replica, consistency, and failover.

Database replication is fundamental to high availability and read scalability. Understanding synchronous vs asynchronous replication is essential for making correct consistency guarantees.

Synchronous replication: Primary waits for at least one replica to confirm the write before acknowledging to the client. Guarantees no data loss on primary failure (zero RPO). Latency penalty: write latency = max(primary + replica network RTT). Used for financial data, anything where losing even one committed transaction is unacceptable.

Asynchronous replication: Primary acknowledges immediately after local write. Replica catches up in the background. Lower write latency. RPO > 0: some committed writes may be lost if primary fails before replica catches up. Replication lag can be seconds to minutes under load. Good for read replicas serving analytics.

PostgreSQL streaming replication: WAL records are streamed from primary to standbys in near-real-time. Synchronous standby: set `synchronous_standby_names`. Asynchronous (default): standby applies WAL as fast as it can. `pg_stat_replication` shows lag per replica.

```

# PostgreSQL replication setup (postgresql.conf on primary)
wal_level = replica
max_wal_senders = 5
synchronous_standby_names = 'standby1' # synchronous replica

# Monitor replication lag
SELECT
  client_addr,
  state,
  sent_lsn - write_lsn AS write_lag_bytes,
  sent_lsn - replay_lsn AS replay_lag_bytes,
  pg_size_pretty(sent_lsn - replay_lsn) AS lag
FROM pg_stat_replication;

# Application: route reads to replica, writes to primary
# SQLAlchemy with separate engines:
write_engine = create_async_engine('postgresql://primary:5432/db')
read_engine  = create_async_engine('postgresql://replica:5432/db')

# Automatic failover: Patroni (PostgreSQL HA)
# Patroni uses etcd/Consul for leader election
# On primary failure: elects new primary, updates DNS/HAProxy
# RTO (recovery time): 10-30 seconds with Patroni

```

Failover and RPO/RTO: RPO (Recovery Point Objective): how much data loss is acceptable. Synchronous replication: RPO=0. Async: RPO=replication lag. RTO (Recovery Time Objective): how long the system can be down. Automatic failover (Patroni): RTO ~30s. Manual: minutes to hours. Design for both explicitly.

Q7: Design a rate limiter — algorithms and distributed implementation.

A rate limiter is a common system design question that tests your knowledge of distributed algorithms, Redis, and correctness under concurrent access.

Where to implement: API Gateway level: limits before traffic hits your services (most efficient). Application level: fine-grained control per endpoint or user tier. Both: gateway for coarse protection, application for business rules.

Fixed window with Redis: Simple: INCR key, set EXPIRE on first increment. Vulnerability: burst at window boundaries (200 requests in 2 seconds spanning a minute boundary).

Sliding window log: Store timestamp of each request in a sorted set. On each request: remove entries older than window, count remaining, allow if below limit. Accurate but memory-intensive for high traffic.

Token bucket with atomic Lua: Most flexible. Allows controlled bursting. Redis Lua script runs atomically — no race conditions between check and decrement.

```
-- Redis sliding window (sorted set approach)
local key = KEYS[1]
local now = tonumber(ARGV[1])      -- current timestamp (ms)
local window = tonumber(ARGV[2])   -- window size (ms)
local limit = tonumber(ARGV[3])

-- Remove entries outside the window
redis.call('ZREMRANGEBYSCORE', key, 0, now - window)

-- Count remaining entries
local count = redis.call('ZCARD', key)

if count < limit then
    -- Add current request with timestamp as score
    redis.call('ZADD', key, now, now .. '-' .. math.random())
    redis.call('EXPIRE', key, math.ceil(window / 1000))
    return 1 -- allowed
end
return 0 -- rate limited

# FastAPI rate limiting middleware
async def rate_limit_middleware(request: Request, call_next):
    key = f'rl:{request.client.host}'
    allowed = await redis.eval(SLIDING_WINDOW_SCRIPT, 1, key,
        int(time.time()*1000), 60000, 100) # 100 req/min
    if not allowed:
        return JSONResponse({'error': 'rate limited'}, status_code=429,
            headers={'Retry-After': '60'})
    return await call_next(request)
```

Distributed rate limiting: Multiple API server instances share a Redis cluster. All rate limit checks go to Redis — the single source of truth. Use Redis Cluster for HA. For extremely high traffic, use approximate rate limiting with local in-memory counters and periodic sync to Redis (eventual consistency, may allow slight over-limit).

Q8: Design a real-time chat system — WebSockets, fan-out, message ordering.

Chat systems test your understanding of WebSockets, pub/sub, message persistence, and the fan-out problem (one message → many recipients).

WebSocket vs polling: Polling: client asks 'any messages?' every N seconds. Wasteful at scale. Long polling: client waits up to 30s for a response. Better but still HTTP overhead. WebSocket: persistent bidirectional connection. Server pushes messages instantly. The right choice for real-time chat.

Architecture: User connects WebSocket to a Chat Server. Multiple Chat Servers, load balanced. A user and their contacts may be on different servers — so servers must communicate. Use Redis pub/sub as the inter-server message bus.

```
# Message flow for 1-1 chat
```

```
User A (Server 1) sends message to User B (Server 2):
```

1. User A sends message over WebSocket to Server 1
2. Server 1 saves message to DB (PostgreSQL/Cassandra)
3. Server 1 publishes to Redis channel: 'user:<B_id>'
4. Server 2 is subscribed to 'user:<B_id>' (B is connected to Server 2)
5. Server 2 receives from Redis, pushes to User B over WebSocket

```
# Message schema
```

```
CREATE TABLE messages (  
  id          UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  conversation_id UUID NOT NULL,  
  sender_id   BIGINT NOT NULL,  
  content     TEXT NOT NULL,  
  created_at  TIMESTAMPTZ DEFAULT NOW(),  
  delivered_at TIMESTAMPTZ,  
  read_at     TIMESTAMPTZ  
);  
CREATE INDEX idx_messages_conv ON messages(conversation_id, created_at DESC);
```

```
# Presence (online/offline)
```

```
# User connects: SET user:<id>:server <server_id> EX 30
```

```
# User sends heartbeat every 20s: refresh TTL
```

```
# TTL expires = user went offline
```

```
# On message: check Redis for recipient's server
```

```
# If key exists: they're online on that server -> push via Redis pub/sub
```

```
# If key missing: they're offline -> store message, push notification
```

Message ordering: Within a conversation, use the DB's auto-incrementing ID or a sequence number. On reconnect, client sends `last_seen_message_id` — server returns all messages with ID > `last_seen`. This handles missed messages during disconnection.

Group chat fan-out: For group with 1000 members: publishing one message to 1000 Redis channels is fine. For WhatsApp-scale groups (1000s of members, millions of groups): fan-out is expensive. Solutions: offline inbox per user (write the message once, recipient fetches on connect), or Kafka for async fan-out processing.
